

availability.

1. The product ecosystem is evolving in a varied fashion through integrations of existing technology stacks, and we are coming upon more specialized data needs.

1. The main GitLab.com Postgres database is intended for OLTP (Online Transaction Processing) use only. Especially at scale, OLAP (Online Analytical Processing) will introduce significant degradation on the database, particularly with regards to cluster replication.

In general, if the codebase is part of the main RoR application, more than likely it already uses the main PostgreSQL database, as the schema is managed through RoR and the code probably has fairly tight coupling with the Rails code base. New entities have to be managed carefully, however, as capacity planning concerns must be addressed in order to ensure availability. The Development Database Group and the Infrastructure teams are the best sources of information in this regard, which also applies to Redis.

#### Exception

Due to scalability concerns, we are currently executing the first functional decomposition of the main database by moving the CI tables to a separate cluster. Although the CI related code base is part of the main RoR application, the CI tables account for ~40% of the overall size and roughly 50% of writes of the main database. Data coupling with the rest of the database is relatively loose, so we are able to ensure data consistency between the two databases while making minimal changes and iterating on new techniques that we build (Loose Foreign Keys).

In general, a separate cluster will only be supported if the size of the dataset and its corresponding transaction rate requirements are large enough to merit the cost of operating it and the added complexity of developing code to support it. Initially, this will likely reside in a separate logical database as part of the main cluster.

#### Other use cases

##### Relational (PostgreSQL)

If the application is not part of the main RoR application code base (for instance, Container Registry, Praefect, Geo Tracking Database), then a separate database is the only option. Whether this is logical or belongs in an independent cluster will come down to scale.

| Code base | Transaction Rate | Storage       | Database         |
|-----------|------------------|---------------|------------------|
| -----     | -----            | -----         | -----            |
| main RoR  | Low, Medium      | Low, Medium   | Main database    |
| main RoR  | High             | High (>1TB)   | FD database      |
| Any       | Low              | Low (<200GB)  | Logical database |
| Any       | Medium           | Medium (<1TB) | Logical database |
| Any       | High             | High (>1TB)   | Separate cluster |

#### New data store engines

There will be times when currently supported data store engines will not be able to meet application requirements and a new data store engine must be introduced in the environment. These should be extremely rare, but will generally come courtesy of integration effort of new technology stacks into our environment (for instance, acquisitions), or when the data needs are

very specialized (for instance, time series).

Due diligence should include involving the Infrastructure team as early as possible, since supporting new technologies will require a variety of work in terms of training, deployment, configuration, maintenance (upgrades), troubleshooting, etc.

---

## SECTION: engineering/architecture/guidelines/data\_lifecycle/\_index.md

## Introduction

Data doesn't disappear; it accumulates and it will bury your application.

Data an application generates has a built-in cost that compounds over time. Without deliberate lifecycle management, unbounded data growth leads to performance challenges, scalability limitations, and increased cloud spend. Further, customers develop dependencies on historical data that make retroactive policies very difficult to implement.

Managing data bloat after the fact is expensive and difficult. Thinking through an application's data lifecycle at the design phase is critical to developing performant and scalable applications.

## Why?

Without proper data lifecycle planning, applications accumulate data indefinitely, leading to several serious consequences:

- Performance Degradation: Database performance deteriorates as tables grow, leading to issues such as plan flipping, and increased query latency due to TOAST storage.
- Escalating Cloud Costs: Unmanaged data growth leads to increased storage costs and excessive compute resources needed to process bloated datasets.
- Operational Complexity: Altering the database like adding indices and columns become increasingly difficult.
- Compliance Risks: Failure to properly manage data retention violates regulations like GDPR that mandate data minimization and limited retention periods.
- Customer Dependency Trap: Without clear lifecycle policies established from the beginning, customers may develop dependencies on historical data, making it nearly impossible to implement retention policies later without disrupting user experiences. This is a version of Hyrum's Law in the context of data.

## Core Data Lifecycle Phases

All application designs must account for the following data lifecycle phases:

### 1. Data Creation

When designing data models, engineers must categorize each data type according to its purpose and anticipated lifecycle:

- Purpose Classification:
  - Operational: Data required for core application functions
  - User-Generated: Data created and owned by users
  - Derivative: Data generated for performance optimization
  - Metadata: Data about system operations and events
- Critical Questions:
  - How is this data generated? (Automated, user input, derived)
  - What business function does this data serve?
  - Who owns this data? (System, user, third-party)
  - What is the expected frequency and volume of data creation?
  - What are the limits that will be set on how much data is allowed to be generated?

## 2. Data Aging

All data models must include explicit consideration of how data value changes over time.

The following diagram shows different phases of data as it ages.

mermaid  
flowchart LR  
A[Fresh] --> B[Recent]  
B --> C[Aged]

- Value Degradation Analysis:
  - Define how rapidly each data type loses operational value
  - Establish criteria for determining when data transitions to "aged" status
  - Example: Pipeline results from merge requests closed more than a year ago has less value than those for an open merge request.
- Critical Questions:
  - How quickly does this data type become stale or less relevant?
  - What's the threshold age at which we should turn longer retention into a paid feature?
  - Are there regulatory reasons to maintain data beyond its operational usefulness?
  - Can aged data be stored more cheaply?

## 3. Data Offloading

By default, all data should have a data retention policy. Any data that needs to be kept around must have a strategy to offload to cheaper storage.

- Tiered Storage Strategy:
  - Define specific thresholds and triggers for data migration

- Implement tablespace partitioning or object storage to facilitate seamless offloading
- Design query patterns that work efficiently with offloaded data
- Critical Questions:
  - What storage tiers are appropriate for this data type?
  - What access patterns will still be needed for offloaded data?
  - How will offloaded data be indexed and retrieved when needed?

#### 4. Data Deletion

By default, all data must have a retention policy for old data to be removed.

- Deletion Framework:
  - Implement automated processes for identifying deletion-eligible data
  - Create audit trails for deletion actions to demonstrate compliance
  - Consider selective archiving capabilities as paid features
- Critical Questions:
  - What is the maximum useful retention period for this data type?
  - Are there regulatory requirements mandating deletion (e.g., GDPR right to be forgotten)?
  - What retention extensions should be offered as premium features?

---

## SECTION: engineering/architecture/guidelines/data\_lifecycle/data\_retention.md

#### Overview

Data retention is a critical aspect of feature development at GitLab. As we build and maintain features, we must consider the lifecycle of the data we collect and store. This document outlines the guidelines for incorporating data retention considerations into feature development from the outset.

#### Why data retention matters

- System performance: Time-based data organization enables better query optimization and efficient data access patterns, leading to faster response times and improved system scalability.
- Infrastructure cost: Strategic storage management through data lifecycle policies reduces infrastructure costs for primary storage, backups, and disaster recovery systems.
- Engineering efficiency: Designing features with data retention in mind from the start makes development faster and more reliable by establishing clear data lifecycles, reducing technical debt and faster data migrations.

#### Guidelines for feature development

##### 1. Early planning

When designing new features, consider data retention requirements during the initial planning

phase:

- Document the types of data being persisted. Is this user-facing data?  
Is it generated internally to make processing more efficient?  
Is it derived/cache data?
- Identify the business purpose and required retention period for each data type.
- Define the product justification and customer usage pattern of older data.  
How do people interact with older data as opposed to newer data?  
How does the value change over time?
- Consider regulatory requirements that might affect data retention (such as Personally Identifiable Information).
- Plan for data removal or archival mechanisms.

## 2. Design for data lifecycle

Features should be designed with the understanding that data is not permanent:

- Avoid assumptions about infinite data availability.
- Implement graceful handling of missing or archived data.
- Design user interfaces to clearly communicate data availability periods.
- Design data structures for longer-term storage that is optimized to be viewed in a longer-term context.
- Consider implementing "time to live" (TTL) mechanisms where appropriate, especially for derived/cache data  
that can be gracefully reproduced on-demand.

## 3. Documentation recommendations

Each feature implementation must include:

- Clear documentation of data retention periods (on GitLab.com and default values, if any) and business reasoning/justification
- Description of data removal/archival mechanisms.
- Impact analysis of data removal on dependent features.

## Implementation checklist

Before submitting a merge request for a new feature:

- ☐ Document data retention requirements.
- ☐ Design data models with data removal in mind.
- ☐ Implement data removal/archival mechanisms.
- ☐ Test feature behavior with missing/archived data.
- ☐ Include retention periods in user documentation.
- ☐ Consider impact on dependent features.
- ☐ Consider impact on backups/restores and export/import.
- ☐ Consider impact on replication (eg Geo).

## Related links

- Large tables limitations

---

## SECTION: engineering/architecture/practice/\_index.md

- Scalability
- Security Architecture Principles

---

## SECTION: engineering/architecture/practice/scalability/index.md

Scalability is a strategic practice. We need

A framework to evaluate technical scalability-related decisions in context  
Best practices guidelines to lead our scalability design and implementation

The framework

Frameworks help contextualize our thinking around a problem by breaking it into manageable conceptual parts, providing vantage points from which we can evaluate choices in both the present and the future. Our design becomes more predictable, our discussions more focused, our decisions likely more accurate and correct. We can make tradeoffs more visible, and, hopefully, easier to understand and reason about.

Swimlanes and the Scale Cube are widely adopted models to manage both fault isolation and scalability in a structured fashion.

Swimlanes

A "swim lane" or fault isolation zone is a failure domain. A failure domain is a group of services within a boundary such that any failure within that boundary is contained within the boundary and the failure does not propagate or affect services outside of the said boundary.

The Scale Cube

The Scale Cube is a model for segmenting services, defining them and scaling products. It also creates a common language for teams to discuss scale related options in designing solutions.

Under this model, the axes represent the following scalability strategies:

X-axis: cloning

cloning of services and data without bias (i.e., "like" workers, duplicating entire datasets)

Y-axis: componentization

separation of work responsibility based on type of data and/or work performed

Z-axis: federation

separation based on customer or requestor bias

These are well-known strategies we are intuitively familiar with. The power of formally adopting the framework lies in that we can now contextualize these strategies and potentially consider several moves ahead in a systematic fashion. The following figure (source: <https://upload.wikimedia.org/wikipedia/commons/5/5f/ScaleCube.png>) depicts these axes:

The Scale Cube model starts at coordinates [0,0,0], where a single instance runs on one system: scalability is entirely dependent on the compute resources associated with the system, and it is scaled by adding more and faster computing resources (CPU, memory, disk). The limiting factor becomes the largest available computing resource, closely followed by cost.

The Scale Cube can be applied to any component in all iterations.

Example: Postgres current state

GitLab.com is currently running at [1,1,0]:

X-axis: The database is composed of N fully replicated instances (a primary RW with several RO secondaries), each of which contains a duplicate of the entire dataset. This configuration can be scaled vertically for the RW workload by adding more hardware, and horizontally for the RO workload (increased query capacity) by adding more replicas. Two limiting factors will play a role:

- RW capacity is directly proportional to hardware capacity (vertical scaling)

- RO capacity will eventually run into the limits imposed by replication lag (CAP will play a role)

Y-axis: We are already migrating some data (diffs) off the database through componentization, albeit without creating a service

A recent analysis indicates our database capacity is well beyond the 12-month range (and estimate that will be updated weekly). As we ponder scalability options, the question is whether the next iteration implements [1,2,0] or [1,1,1]. Current proposals essentially advocate for the latter strategy, but there is a case to be made for choosing an entirely different direction.

## Scalability Best Practices

This section collects scalability best practices, gathered over time through experience (ours and others'). They are guidelines, not rules, that we apply to work through these problems in a structured fashion, leveling the cognitive playing field.

### [AMF] Always Monolith First

The first rule of scalability is: don't.

This is a variation on a well-known theme: functionality first, avoid premature optimization. Always start in the monolith, and only extract from it when there is a good reason to do so. A "good reason" is always a data-driven decision (via KPIs). Doing so will avoid unchecked proliferation of "services" inside the application, which is in itself a significant problem (unmanageable dependencies, orphaned or neglected services, etc). As a guardrail, we can set a

strict limit on the number of "services" allowed within the application, and it should be organizationally difficult to raise that limit. It is important to note the implication is that the service may not be optimized and perform as well as the rest of the product. It should still be reliable or have a published error budget and performance target so that users can make an informed choice.

Remember, however, that scalability is a strategic practice, so keep the Scale Cube in mind: consider the relationships created by new entities carefully and within the scalability framework: what would happen if an entity needed to be componentized or federated in the future?

[SSI] Serialize Scalability Iterations

Never attempt to scale across two axes at the same time.

[CBF] Componentization Before Federation. Most of the Time

Componentization breaks down a system into logical, interconnected components. Federation refers to [TODO: definition].

Componentization provides scale by creating headroom throughout the freed up resources extracted from the system. It also allows us to scale them relatively independently of other components, as locally optimize and scale individual components as necessary: in general, we need only worry about appropriately scaling downstream dependencies, so as to not overwhelm them. In extreme cases, we may choose an entirely different class of datastore (for instance, we may determine that storing comments in a document database is more effective than doing so in a relational database; we are already doing this by migrating diffs off of Postgres into object storage).

Componentization is not free:

Data comes from a separate subsystem and thus a separate runtime scope, so data mappers and data composers are required to compose the data we need. The application must now take over some of those responsibilities, which had been previously delegated to the datastore.

Internal latencies will increase, so caching enters the equation; cross-indexing may be required

Failure modes become more diffuse: a complete component failure may be straightforward to detect, but when results are returned, we must be able to ascertain said results are the full set of results expected, or be able to understand when partial results are being returned.

Componentization can (and should) adhere to efficient iteration. Componentization entails breaking up GitLab into logical components such as repositories (already somewhat there: Gitaly, which has in fact also been federated through hashed paths), issues, comments, pipelines, etc. We must place a sensible cap on the number of components allowed (AMF). Federation entails vertical axes (tenants, for instance).

One problematic issue with federating before componentizing is that, long-term, we end up attempting to scale federated members vertically. While in some cases this may work well, we

may reach a point where we need to componentize all federated members, a maneuver that adds complexity to the process.

### Most of the Time

There are times, however, when federating first makes sense, especially when we can provide a low impact MVC that creates customer value or can lay the foundation of future iterations. However, these should be limited to cases where the scope is at its shallowest (see MVF).

### [MVF] Minimum, Variable-depth Federation

We iterate efficiently, so we try to operate on the minimum scope possible: only implement the shallowest federation in any given iteration, as measured in terms of the number of affected layers in the stack and the distance between said layers. As a very practical matter, federation never cuts across the entire stack in a single iteration: use it whenever, wherever, and however it makes sense to do so, but minimize the scope. Variable-depth allows us to be selective in its application (which is aligned with boring solutions and MVC),

### Examples

Federation may be as simple as having separate DNS endpoints into GitLab.com for certain types of customers, where types may be defined across any number of variables: a business entity (i.e., company Acme), geolocation (eu.gitlab.com, gitlab.com.de, or gitlab.cn). What lies behind is entirely up to us.

### [NFR] Never Federate Relationships

Never federate relationships. Relationships are not true entities, even if they sometimes take the form of entities. For examples, namespaces are relationship entities: their sole function is to function as a relationship bridge. Compare to true entities, like projects, users, or issues.

### [NFE] Never Federate Entities

Much like we should never use an entity's attribute as a primary key, we should never federate entities to create intrinsic swimlanes, which is to say, never use an entity's attribute identifier as a swimlane identifier. Federate using an entity's attribute allowing for multiple entities to share a swimlane. In the extreme case,  $N = 1$ , but not by making the entity's identifier the swimlane's identifier.

### [FCC] Federation is a Customer Choice

We should never force customers to embrace federation, especially existing customers. In particular, federation on edge layers, which tend to be those closest to the customer, can be extremely disruptive (e.g., front doors). People tend to hardcode names in configurations and scripts. This is very true for us, where millions of people have cloned repositories with Git configurations pointing to GitLab.com.

### [CTF] Compliance Through Federation

Since federation is based on customer or requestor bias, it enables the activation of features along those boundaries for customers that want a certain feature (geolocated data). Regulatory compliance is thus enabled through federation.

#### [CTL] Componentizing on Time Locality

Sometimes, it makes sense to split a given component in terms of time. Pipelines and related data, for instance, is one example of a potential time-split. Perhaps we keep all configuration and schedules in a fast, relational database, but we move 6-month old data to a separate, perhaps slower database.

#### Workflow

#### Performance Indicators

Performance indicators are scalability-related instrumentation that enables us to understand component and application scalability limits. They're currently being worked on by the Scalability Team:

<https://docs.google.com/document/d/1IZ7RKtv7yCkV7MVx-7UfZZMvEB9IzLrrFDUw0oVFxXA/edit#heading=h.ibm29qjhrqeb>  
<https://gitlab.com/gitlab-com/gl-infra/scalability/-/issues/382>

Performance indicators allow prioritization of scalability work.

#### Articulate Objectives

It is important to articulate scalability objectives before embarking on scalability work. It needs not be a detailed specification: simplicity rules, and the objectives can be refined over time. (Note: PIs are key) Most of the time, we will not be able to accurately predict the exact effects of a given scalability iteration. Estimates are therefore ok (but show the math).

#### Scope the Solution

What potential solutions(s) would meet the objectives? Use the Scale Cube and Best Practices to scope it (them), and evaluate what future steps might look like (step-functions). Engage through the Architecture Workflow.

---

## SECTION: engineering/architecture/workflow/index.md

As engineers at GitLab, we lead the evolution of software, constantly working to find the right balance between proactive work, reactive work, and innovation. We strive to determine what work is important and what work is not, leveraging knowledge from those that know the most about GitLab, and empowering people to work on things that make everyone more productive. Experimenting and innovating are core to how we work, and we focus on collaboration, results and iteration to achieve our goals.

With growth, however, comes complexity. An organic approach to our work sometimes requires help to ensure we are most efficient. Help may be in the form of validating our technical approaches, ensuring organizational alignment across teams and departments, and driving priorities to key decision makers. Technical Engineering Leaders take on the task of helping engineers through these challenges. The Architecture Design Workflow is intended to provide guidance, influence amplification, iteration framework and additional visibility to drive the solution of complex problems both technically and organizationally.

## Design Documents

Design documents are the primary artifact that the workflow revolves around. They are version controlled documents that are released in this handbook user-facing documentation and you can find a list of published ones there too.

Some of the design documents also live in separate repositories when a design doc has to be confidential or when custom project configuration is needed, see GCP integration design doc as an example.

Long-term iterations, longer than a single milestone, either on features or maintenance tasks, are challenging because it is easy to lose focus, consensus, conceptual integrity, architectural consistency, or alignment in why and how we are doing something.

A design document describes a technical vision, a set of principles and key architectural decisions that will guide implementation, as we move forward. It acts as guardrails to keep team aligned. Design documents get constantly updated with new insights and knowledge, after every iteration, to become even more useful with time.

You can start with a design document that is one paragraph long, and evolve the content as you move forward with your exploratory work, depending on what you learn along the way. Design documents are not supposed to be complete and detailed blueprints written upfront before we start implementation.

Why are design documents tracked in merge requests?

Design documents are tracked as version controlled artifacts. This enables anyone to propose changes in the form of merge requests. The merge requests can be reviewed and approved before merging. Engineers usually provide feedback in code review process by leaving comments in merge requests' diffs. We are using the same process here. By doing so we can ensure that:

- there is always a single document that represents the current state of a given proposal
- you do not have to traverse multiple issues or threaded discussions to understand the direction
- feedback can be given and applied in the form of concrete suggestions and comments
- proposals / changes / decisions are being made in merge requests using "design as code"

## workflow

Do I need to use Architecture Design Workflow?

Using the workflow is recommended for changes that meet any of the following conditions:

- requires coordination across multiple functions, teams, departments
- could impact overall system stability, availability or performance
- multiple team members are being involved across multiple milestones
- changes GitLab in a significant way
- impacts the operation of GitLab substantially
- introduce special handling across distributions and deployments
- adds a new service outside of the rails monolith, or an additional data source.

Invoking this workflow is unnecessary if:

- fixing a flaky test
- minor refactoring of code
- small performance improvements
- upgrading versions of dependencies

Invoking the architecture design workflow is also not necessary when you are doing a complex thing that you have a lot of experience with, and you've done at GitLab many times before. In such case, then perhaps there is no benefit in involving a Coach and using the workflow.

Please use a pragmatic approach when deciding whether to use the workflow or a regular lightweight design process by considering the cost (process overhead) / benefit (guidance, coaching, visibility) ratio.

## Architecture Design Workflow

### Summary

1. Start writing a design document! Depending on whether the content can be considered SAFE or not you may want to do that in a private space first. If you don't know what content to start with you can use a template.  
You will find a couple of suggestions there and a markdown front matter we use for status tracking.
1. Open a merge request  
if you have not done it already and if it is SAFE to do so.
1. Post a link to your design document, with a brief description of it, in the internal architecture channel on Slack for additional visibility and transparency.
1. If your design document describes a complex effort that will span multiple milestones, you may want to involve a Coach Engineer: usually a Principal+ Engineer who will support you throughout process of describing a technical vision in your design document.

1. Work with the stakeholders and domain experts to get your design reviewed and commented on. Design docs don't need to be comprehensive nor complete on the first iteration. They can be refined and enriched with details in iterations, as we discover important design aspects, fundamental design areas, key decisions. We advise to get our initial design doc merged as soon as possible.
1. Start exploration, research and implementation! Start writing code as soon as possible!
1. Iterate on the design doc in parallel as you discover new details, make fundamental design decisions, learn more about how you want to proceed with the implementation refine the content of the document frequently.
1. Take a look at existing design docs and learn from past experiences of engineers who have been successful with using the workflow.
1. Improve the workflow if you learn something that will make us all more productive next time!

## Roles

### The Author

As the original author you are the primary DRI responsible writing the design doc.

The Author is a DRI responsible for driving the process of writing a design document. They can collaborate with a Coach, Engineering Management Leader, Product Management Leader, Domain Experts, Functional Experts during the process.

### The Coach Engineer

Coach is a Principal+ Engineer, who has been already involved in work on the complex technical initiatives, who can guide the author throughout the process as a mentor and a coach.

The purpose of involving a coach in the process of writing a design document is to allow people that know most about GitLab to share their knowledge and perspective on introducing complex architectural changes, help navigate organizational challenges, ensure the proposal is aligned with our roadmap, and help management Engineering Leaders prioritize the work.

Involving a Coach Engineer is optional, but we strongly advise to involve one if:

1. Accepting a proposal outlined in the design document means that we will need to spend 6+ milestones on the implementation.
1. Multiple people from a team or a department will need to be involved in the implementation for a longer period of time.
1. It is a cross-functional or foundational initiative, where a single team will not be enough to implement a vision described in the design document.
1. A team struggles to describe the proposal in a design document and would greatly benefit from involving a Coach to provide guidance and mentoring.

## The Engineering Management Leader

The Coach may be able to help you identify the right management Engineering Leader to evaluate the proposal. Managers are key decision-makers, and, ultimately, will help to navigate the organizational complexities to get your proposal approved and funded.

## The Product Management Leader

The Engineering Management Leader and a Coach might be able to help you to identify the right Product Manager to collaborate on the proposal. PMs are the decision-makers that will help to include your proposal in the stream of work that is always in-flight. PMs can also help with funding your proposal if they believe that we need to hire new people to get it done or to invoke other processes to find people who can work on it.

## Domain Experts

### Domain Experts

are engineers with a deep understanding of one or more particular areas. Domain Experts:

1. Help to ensure conceptual integrity of the features and changes their groups / stages / sections are working on
1. Help to collaborate with EMs and PMs and other Engineers to ensure the quality of work done in their area of interest
1. Help to plan and draft necessary architectural and conceptual changes that will become a leverage in their area of interest

A Domain Expert is an engineer, usually an individual contributor, who knows most about specific aspects of the codebase and a domain in the area of proposed changes, but might still lack the deep understanding of the process behind introducing complex architectural changes, hence the collaboration between a Domain Expert and a Coach might be very useful.

## Functional Experts

Functional Experts are engineers with deep knowledge across specific functional areas, which include Security, QA, Database, and Infrastructure. You should always consider involving these functional experts during the creation of a design document, so that we generate awareness early in the cycle and so that they can provide their input.

## Design doc structure

A design doc is usually divided into multiple sections:

1. The introduction
1. Business objectives
1. High-level overview

1. Goals and key results
1. Current state pointer
1. Fundamental design areas
1. Key design decisions
1. Implementation details
1. Closing summary and references

As an author you decide which sections you want to include in your design doc. Merged design documents will be published in our public handbook.

## Introduction

The introduction is a high-level executive summary explaining why we are doing something and what is our approach to getting the thing done. The audience of the here is very broad, so it is important to avoid excessive technical language.

## Objectives

If the architecture design workflow is being used it means that what we want to accomplish is going to be non-trivial and presumably expensive for the company. An investment like this should have a solid business justification, we should know what is going to be a business impact. It is recommended to add a note about expected business results to a design doc.

## Overview

The main design doc's content is usually a high-level overview of why and how we want something to be implemented.

There are many ways in which a design doc can be written. We encourage taking a pragmatic approach to writing a design doc: choose a way that works well in your case. One of the frequently used ways is to keep the overview section short and keeping it up-to-date. Find existing docs that have proven to be useful to better understand how it can be written.

## Decisions

We strongly recommend documenting key decisions made about fundamental aspects of the design in the design doc. These decisions are going to become important checkpoints during the implementation, and will provide more clarity around the direction to newcomers.

To document a decision a lightweight process can be used:

- Identify fundamental design problems and note them down.
- Breakdown complex decisions into smaller ones, if needed.
- Describe the context on what we need to make a decision on.
- Document benefits, trade-offs and alternatives considered.
- Document why a specific solution was chosen.

You can use a lightweight Architectural Decision Record (ADR).

See an example [here](#).

We recommend to add ADRs as subpages, and link them from a Decision section on the main page of the design doc. An ADR usually has a few key sections: context, decision, consequences, alternatives considered.

An example of an important design area could be "Client-server communication protocol", while an example of decision could be: "Use Protocol Buffers as data serialization format".

Team members often write immutable ADRs. If a decision has to be changed, we can note that it has been superseded and create a new ADR with the new decision. This helps to reduce the amount of work required to keep a design doc up-to-date. See an example in [GCP integration design doc](#).

Sometimes it also makes sense to document known engineering areas in which we haven't made any decisions yet, but we know we will have to. We call them "fundamental design areas".

### Implementation Details

As we move forward with implementation and iterate on a project, we continuously incorporate feedback gained after each of the iterations and we often add it into the design document itself. Technical details can go into subpages to keep the main page of the design document (with the overview) short and easy to read. Implementation details can be also extracted into issues / epics.

### Getting things done

Each design doc requires DRIs. DRIs decide on how to proceed with the design doc, how to get it implemented, using known organizational tools and frameworks, like a Working Group.

Typically we either proceed with a WG or with a dedicated team which will focus on executing on a design document.

### Amplification

Anyone can propose a change they believe we should work on. When these changes turn out to be too intricate for a single individual contributor to handle (complex backstage improvements, architectural changes, productivity or efficiency improvements), or they span multiple iterations or teams, it may be helpful to use the Architecture Design Workflow to get the proposal captured in a design doc. The author of the proposal can collaborate with a Coach Engineer, who will involve the right people to make sure that the proposal is well described and gets considered for implementation.

We recognize the challenge of implementing complex changes or features, over many months or even years. It is difficult to start such a work, fund it in the

long term and avoid disruptive distractions as the implementation moves forward.

Design documents are often written by individual engineers, yet these documents usually describe far-reaching visions. Implementing such a vision takes time and might require funding. The Architecture Design Workflow has been built to better support teams in getting this kind of work done. There are a few associated processes, established to increase the likelihood of a success.

One of processes designed to help is a monthly Architecture Evolution Sync meeting with Engineering Fellows and Engineering Leadership, among others. The purpose of this meeting is to:

- Increase visibility and awareness of key design documents.
- Coordinate large initiatives across the organization.
- Provide status updates about the most important initiatives.
- Receive guidance regarding staffing and funding.

### Iteration

The Architecture Design Workflow does not dictate how much time you should spend on the design phase, before you start writing production code. It is recommended to start writing a design doc and start writing code at around the same time, but in certain cases it might be beneficial to spend more time on the careful design upfront, followed by a set of feasibility studies before the actual implementation starts. In the former case a design doc is a technical artifact which evolves in parallel to us building the actual solution. In the latter case, we make an implementation proposal before the implementation starts, and then refine the design doc as needed as we move forward. In either case the design doc will keep changing; it is rarely possible to write a design doc and then to implement it without changing something during the process. Use a pragmatic approach here, involve Coach Engineers if needed. There is no right or wrong here.

### Finally

When the work is completed, design documents no longer represent a forward-looking vision, instead the content describes the work that has been done and decisions that has been made. As such, a design document should be updated to become more useful as a knowledge-sharing artifact, helping new engineers and contributors to get familiar with design decisions and architectural choices more quickly. If you are using an immutable architectural decisions log (ADRs) updating the design doc should not require a lot of effort. Alternatively you can archive a design doc if using it as documentation does not serve any meaningful purpose.

### Design Document Status Definitions

To provide clarity and consistency across the architecture workflow, the following statuses are used to indicate the current state of each design document:

- proposed: The design has been drafted and is awaiting review or feedback. It has not yet been formally accepted.
- accepted: The design has been reviewed and approved. Work may begin based on this design.
- ongoing: Work based on the design is actively being implemented.
- implemented: Work has been completed and the design has been fully implemented.
- rejected: The design was reviewed but not accepted. It will not be implemented in its current form.

These statuses help track the lifecycle of architectural designs and provide visibility into their current state.

---

## SECTION: engineering/careers/\_index.md

## The Three Components of Career Development

There are three important components of developing one's career:

### Structure

Team members who are (or want to be) on track for promotion should be engaged in a career coaching conversation with their manager. Some basic information about this process can be found in the People Ops handbook. Specific coaching plan templates are listed here to help start the conversation:

- Senior Engineer
- (Others to come)

We want to build these documents around the career matrix for Engineering. Since this career matrix is still being developed, these documents are currently based on the job family requirements.

The first career matrix being developed is for Individual Contributors in Engineering.

When a team member is deemed ready for promotion, their manager should follow the promotion process outlined by People Ops.

Remember that the coaching process helps team members understand what they need to do in order to prepare for a more senior position on the team. The promotion process documents what an engineer has already done to deserve a more senior position on the team. The two processes are related, but they are not substitutes for each other.

### Individual Initiative

Most career opportunities involve stepping into a position of informal or formal leadership. As such, the initiative of the individual is a necessary component, as well as a qualification. However, for the sake of inclusion, we do ask that

managers periodically bring up the possibilities for advancement so that individuals know the avenues available to them and receive encouragement to pursue them. Managers and individual contributors should strive to have a career development conversation at least once a quarter.

## Opportunity

The GitLab Engineering career track offers a full path of opportunity for individuals to contribute.

There are scenarios where a position must become available before a promotion can occur. On the engineering track, there must be a position of need to be promoted from a Senior Engineer to either a Staff Engineer or an Engineering Manager, depending on your chosen path. Progression beyond these types of roles (e.g. Senior Engineer) are not guaranteed by time in role. There must be a justifiable need for the role and the candidate must have some level of demonstrated ability to thrive in these roles to be considered for the new position.

These next level opportunities are not guaranteed to be available and should not be viewed as career limiting. Again, using the Senior Engineering role as an example, is a critical role within GitLab. It is an accomplishment to thrive at the Senior level and should be celebrated as such.

## Mobility Principles

The Engineering Mobility Principle is an add-on to the company's mobility guidelines. For cases not covered in this section, please visit the People Ops Internal Mobility guidelines

Career mobility is a part of career development, we support and encourage development and opportunities of our team members while maintaining or making conscious choices of impact to business objectives, current business team capacity, and budget constraints. The following steps must be followed

- Internal mobility should not be an easier path than a promotion for level change
- All new roles will follow inclusive hiring guidelines for posting and making them available to all potential internal candidates (must be open for a minimum of 5 days before any offer is made)
- All new roles have a recommended level that is determined by the expected return on investment for the given budget. Managers who wish to consider candidates above or below the given role may offer an interested internal candidate, the following two options:
  - Re-level the role with a business justification for how this change will benefit the company.
  - A template (google document) has been provided, please create a copy and fill in the business justification.
- After evaluating the potential readiness of the candidate to meet the expected criteria within a given time period, accept the candidate for an interim period, up to 6 months, and evaluate readiness and impact of level change at the end of the interim period.
- Transfers between career paths (IC->EM, EM->IC) the person should be moved into an available role on an interim/acting basis for 4-6 months before the change is made permanent with a promotion/evaluation rationale based on performance
- All role changes will be reviewed by R&D leadership on a weekly basis. This review will

include impact on roadmaps and mitigation of the budget impact if the change requires additional budget to accommodate the request.

- All internal transfer candidates should have an aligned transition and communication plan to address impact on the previous team, including finalizing a plan for backfill and completing any milestones that are already underway.

### Individual Contribution vs. Management

Most important is the fork between purely technical work and managing teams. It's important that Engineering Managers self-select into that track and don't feel pressured. We believe that management is a craft like any other and requires dedication. We also believe that everyone deserves a manager that is passionate about their craft.

Once someone reaches a Senior-level role, and wants to progress, they will need to decide whether they want to remain purely technical or pursue managing technical teams. Their manager can provide opportunities to try tasks from both tracks if they wish. Staff-level roles and Engineering Manager roles are equivalent in terms of base compensation and prestige. Learn more about what it means to be a Staff-level engineer at GitLab in an Engineering IC Leadership page.

### Trying the Management Track

It's important that team members interested in the people management track have opportunities to try it out prior to committing themselves. Managers can provide multiple opportunities to Senior and Staff Engineers who want to consider moving into an available or upcoming manager role. Examples include hosting a Group Conversation, acting as the hiring manager for an intern position, or running a series of demo meetings for an important deliverable. These occasions for trying out management functions are useful and can provide good coaching opportunities for the individual looking to move to management. Some engineers may find through these experiences that they are better suited to remain on the technical path, and this gives them the ability to decide before investing a significant amount of time in the transition.

In order to facilitate the transition, we recommend any person moving from an Individual Contributor role to an Engineering Manager role work with their manager to create a focused transition plan. The goal is to provide a concentrated look into the responsibilities and challenges of people management with the understanding that, if the role is not for them, they can return to the IC track. A good example of how to build such a plan can be found in this article. Another good resource for when you are ready for the critical career path conversation is this 30 mins video.

### Temporary Management Positions

We create temporary management positions when there is organizational need, and we imprint these in our company org chart. These may be filled by someone who is transitioning into the role (Interim Manager), experimenting with the role as they work on determining their career path (Interim Manager or Acting Manager) or, someone who is just filling in while we hire and who is not interested in pursuing an Engineering Manager role long term (Acting Manager). This difference should be made explicit with the individual and team members before the temporary role is created. See the types of temporary roles for more information on this distinction.

When someone fills a temporary role they are providing a service to the company, and perhaps getting a valuable career development opportunity for themselves; so, poor performance against those duties would not result in termination. At worst, the person would return to their prior responsibilities. That does not mean, however, that an individual is immune to termination of their employment, for example if they commit a breach of their prior responsibilities or of the company's Code of Business Conduct & Ethics while in a temporary position.

Once you have been designated as filling a temporary management role, the current manager should update all reports to in Workday with a job information change request and create an access level request to grant interim manager access in Workday. Note that direct reports in Workday generally are not moved to Acting managers.

Those interested in these roles must:

- show managerial interest
- have experience in the individual contributor role for which they would be managing
- have been employed at GitLab for at least 6 months and thriving in our remote-only context
- exemplify GitLab Values
- For Acting Manager roles: be subject to a talent review depending on the length of appointment
- For Interim Manager roles: Apply through Greenhouse and go through selection process

Process for selection:

- Upcoming Interim and Acting manager roles will be discussed over the Staff/Department meetings and/or via Department Slack channels.
- Leadership can gather interest from their team members for the upcoming roles.
- Based on the above criteria the assessment will take place and most suitable team member will be selected for the role.

## Types of Interim Roles

### Acting Manager

An Acting Manager is someone who occupies a role temporarily and will move back to their original role after a set amount of time or other conditions. An Acting Manager may be experimenting with the role as a part of determining their career development path, or may be filling in for a vacant role while we hire.

### Interim Manager

When a team member is pursuing a people manager role (Level Manager+) long term we apply an interim period in the following instances:

| Current level                 | Desired level | Previous Management experience (3+ months) |
|-------------------------------|---------------|--------------------------------------------|
| Application of interim period |               |                                            |
| -----                         | -----         | ----- -----                                |
| -----                         |               |                                            |
| Senior                        | Manager       | No   Yes                                   |
|                               |               |                                            |

|                    |  |                         |                        |     |
|--------------------|--|-------------------------|------------------------|-----|
|                    |  | Yes                     |                        | Yes |
| Staff              |  | Manager                 | No                     | Yes |
|                    |  | Yes                     |                        | No  |
| Manager            |  | Senior Manager          | No (managing managers) | Yes |
|                    |  | Yes (managing managers) |                        | No  |
| Manager/Sr Manager |  | Director                | No (managing managers) | Yes |
|                    |  | Yes (managing managers) |                        | No  |

Note that previous management experience can be management experience in a previous organization, or management experience at GitLab. "Acting" role management experience at GitLab can also count towards the 3 month threshold if that acting role was similar in scope and influence as the interim period would have been as evaluated by the hiring manager.

#### Exceptions

For Director+ positions no interim is required.

Exceptions can also be reviewed on a case-by-case basis depending on readiness for team members who have served in acting roles, and who have passed the interview process for a permanent role.

#### Timeline for interim roles

The set timeline is for interim roles to be 3 months, and we recommend that interim periods do not exceed this timeframe.

There might be situations where an interim role does not need to be 3 months but a shorter period. This can happen when either the below success criteria are met before the 3 months pass or the team member has had experience with the new role prior to starting the interim period (for example via an acting role). If this is the case please reach out to your Department Head and People Business Partner to review.

If there is an exceptional case in which the manager feels they need more than 3 months to assess team member readiness in an interim position, they should discuss this with their Department Head and People Business Partner.

For acting roles, there is no set timeline for duration. This can be reviewed on a case-by-case basis.

#### Success criteria for Interim Roles

The main success criteria for interim managers at GitLab is to make one successful hire. The official promotion will not occur before 30 days after that person's hire, so that we can

assess whether the hire was truly successful. If the new hire's success is indeterminate at the 30-day mark, then we will continue to review until a firm decision is made. If the new hire is not successful, that does not mean that the Interim Manager cannot eventually move into the full-time role.

In some cases, it may not be practical, fitting with the interim timeline or headcount planning might not allow for the Interim Manager to make a new hire. In this case the Interim Manager and the interim manager's manager should agree on success criteria based on the requirements of the role. The interim period not exceeding three months should take precedence over not being able to make a hire (so long as other success criteria are in place). Examples of other success criteria are (multiselect):

- Successfully pass an interview(s) with the Leader(s) of the department(s) around team management competencies/scorecard for Manager;
- Successfully pass team member relations case role play with the People Business Partner;
- Performing career development conversations with direct reports successfully;
- Perform a Talent Assessment with the manager's manager to determine readiness for the role;
- Gather 360 feedback via Culture Amp for the manager.

Once the Interim Manager's first new hire has been at GitLab for 30 days, or other agreed success criteria has been met, the Interim Manager and Manager can work together to create and submit a promotion document for approval.

On top of the success criteria we require that before moving into fulltime positions, interim managers begin the Becoming A Manager training. While up to 3 months in an interim position with hands on experience is helpful in determining whether people management track is right for team members, many situations and processes that people managers drive are not covered in this time period. The Becoming A Manager issue helps ensure that potential managers have a fuller picture in terms of policies, processes, and situations managers work through as part of their roles. The Becoming A Manager training is structured to be spread over the course of 3 months. Given that interim periods should last a maximum of 3 months, it may not be viable to fully complete the training before the interim period ends. The requirement is that the training issue is open and well underway before we move to permanent positions, and that it is finished aligned with the 3 month training timeframe.

For more details on logistics of ending an interim or acting period please refer to the promotions and transfers page.

## Roles

### Development Department

```
<div style="overflow-x: auto">
<div style="width: 1800px;">
```

```
mermaid
graph LR;
%% frontend roles
eng:tfe(Intern Frontend Engineer);
click eng:tfe "https://handbook.gitlab.com/job-
```

families/engineering/development/frontend/intern-frontend-engineer";

eng:afe(Associate Frontend Engineer);  
click eng:afe "https://handbook.gitlab.com/job-families/engineering/development/frontend/associate-frontend-engineer";

eng:ife(Intermediate Frontend Engineer);  
click eng:ife "https://handbook.gitlab.com/job-families/engineering/development/frontend/intermediate-frontend-engineer";

eng:sfe(Senior Frontend Engineer);  
click eng:sfe "https://handbook.gitlab.com/job-families/engineering/development/frontend/senior-frontend-engineer";

eng:stfe(Staff Frontend Engineer);  
click eng:stfe "https://handbook.gitlab.com/job-families/engineering/development/frontend/staff-frontend-engineer";

%% backend roles

eng:tbe(Intern Backend Engineer);  
click eng:tbe "https://handbook.gitlab.com/job-families/engineering/development/backend/intern-backend-engineer";

eng:abe(Associate Backend Engineer);  
click eng:abe "https://handbook.gitlab.com/job-families/engineering/development/backend/associate-backend-engineer";

eng:ibe(Intermediate Backend Engineer);  
click eng:ibe "https://handbook.gitlab.com/job-families/engineering/development/backend/backend-engineer";

eng:sbe(Senior Backend Engineer);  
click eng:sbe "https://handbook.gitlab.com/job-families/engineering/development/backend/senior-backend-engineer";

eng:stbe(Staff Backend Engineer);  
click eng:stbe "https://handbook.gitlab.com/job-families/engineering/development/backend/staff-backend-engineer";

%% fullstack roles

eng:tfs(Intern Fullstack Engineer);  
click eng:tfs "https://handbook.gitlab.com/job-families/engineering/development/fullstack/intern-fullstack-engineer";

eng:afs(Associate Fullstack Engineer);  
click eng:afs "https://handbook.gitlab.com/job-families/engineering/development/fullstack/associate-fullstack-engineer";

eng:ifs(Intermediate Fullstack Engineer);  
click eng:ifs "https://handbook.gitlab.com/job-

families/engineering/development/fullstack/intermediate-fullstack-engineer";

eng:sfs(Senior Fullstack Engineer);  
click eng:sfs "https://handbook.gitlab.com/job-  
families/engineering/development/fullstack/senior-fullstack-engineer";

eng:stfs(Staff Fullstack Engineer);  
click eng:stfs "https://handbook.gitlab.com/job-  
families/engineering/development/fullstack/staff-fullstack-engineer";

%% Engineering Manager  
eng:em(Engineering Manager);  
click eng:em "https://handbook.gitlab.com/job-  
families/engineering/development/management/engineering-manager/";

%% engineering leadership  
eng:sme(Senior Engineering Manager);  
click eng:sme "/job-families/engineering/engineering-management/senior-manager-development";

eng:dir(Director of Engineering);  
click eng:dir "/job-families/engineering/engineering-management/director-development";

eng:pre(Principal Engineer);  
click eng:pre "/job-families/engineering/engineering-management/principal-engineer";

eng:dis(Distinguished Engineer);  
click eng:dis "/job-families/engineering/engineering-management/distinguished-engineer";

eng:sde(Senior Director of Engineering);  
click eng:sde "/job-families/engineering/engineering-management/senior-director-development";

eng:vpd(VP of Development);  
click eng:vpd "/job-families/engineering/engineering-management/vp-of-development";

eng:fel(Engineering Fellow);  
click eng:fel "/job-families/engineering/engineering-management/engineering-fellow";

%% career progression  
subgraph Based on Merit & Company Need  
eng:stfe & eng:em  
eng:stbe & eng:em  
eng:stfs & eng:em;

eng:em-->eng:sme-->eng:dir-->eng:sde-->eng:vpd;  
eng:stbe & eng:stfe & eng:stfs-->eng:pre-->eng:dis-->eng:fel;  
end  
subgraph Merit Based  
eng:tfe-->eng:afe-->eng:ife-->eng:sfe-->eng:stfe & eng:em  
eng:tbe-->eng:abe-->eng:ibe-->eng:sbe-->eng:stbe & eng:em;  
eng:tfs-->eng:afs-->eng:ifs-->eng:sfs-->eng:stfs & eng:em;

end

</div>

</div>

## Quality Department

<div style="overflow-x: auto">

<div style="width: 1800px;">

mermaid

graph LR;

subgraph Merit Based

qual:inbeep(Intern, Backend Engineer, Engineering Productivity)-->qual:ibeep(Intermediate Backend Engineer, Engineering Productivity)

qual:jset(Intern Software Engineer in Test)-->qual:iset(Intermediate Software Engineer in Test);

qual:iset(Intermediate Software Engineer in Test)-->qual:sset(Senior Software Engineer in Test);

qual:ibeep(Intermediate Backend Engineer, Engineering Productivity)-->qual:sbeep(Senior Backend Engineer, Engineering Productivity);

qual:ifsecs(Intermediate Fullstack Engineer, Contributor Success)-->qual:sfsecs(Senior Fullstack Engineer, Contributor Success);

end

subgraph Based on Merit & Company Need

qual:sset(Senior Software Engineer in Test)-->qual:stset(Staff Software Engineer in Test);

qual:sset(Senior Software Engineer in Test)-->qual:qem(Quality Engineering Manager);

qual:sbeep(Senior Backend Engineer, Engineering Productivity)-->qual:stbeep(Staff Backend Engineer, Engineering Productivity);

qual:sbeep(Senior Backend Engineer, Engineering Productivity)-->qual:bemep(Backend Engineering Manager, Engineering Productivity);

qual:sfsecs(Senior Fullstack Engineer, Contributor Success)-->qual:stfsecs(Staff Fullstack Engineer, Contributor Success)

qual:bemep(Backend Engineering Manager, Engineering Productivity)-->qual:dqe(Director of Quality Engineering);

qual:qem(Quality Engineering Manager)-->qual:dqe(Director of Quality Engineering);

qual:dqe(Director of Quality Engineering)-->qual:vpq(VP of Quality);

end

click qual:jset "/job-families/engineering/software-engineer-in-testintern-software-engineer-in-test";

click qual:iset "/job-families/engineering/software-engineer-in-testintermediate-software-engineer-in-test";

click qual:sset "/job-families/engineering/software-engineer-in-testsenior-software-engineer-in-test";

```

click qual:stset "/job-families/engineering/software-engineer-in-teststaff-software-engineer-
in-test";
click qual:ibeeep "/job-families/engineering/backend-engineer/engineering-productivity";
click qual:sbeeep "/job-families/engineering/backend-engineer/engineering-productivity";
click qual:stbeeep "/job-families/engineering/backend-engineer/engineering-productivity";
click qual:bemep "/job-families/engineering/backend-engineer/backend-engineering-manager";
click qual:qem "/job-families/engineering/engineering-management-quality/quality-engineering-
manager";
click qual:dqe "/job-families/engineering/engineering-management-quality/director-of-quality-
engineering";
click qual:vpq "/job-families/engineering/engineering-management-quality/vp-of-quality";

```

</div>

</div>

## Support Department

```
<div style="overflow-x: auto">
```

```
<div style="width: 1800px;">
```

mermaid

graph LR;

subgraph Merit Based

```
supe:sa(Associate Support Engineer)-->supe:se(Support Engineer);
```

```
supe:se(Support Engineer)-->supe:sse(Senior Support Engineer);
```

end

subgraph Based on Merit & Company Need

```
supe:sse(Senior Support Engineer)-->supe:stse(Staff Support Engineer);
```

```
supe:sse(Senior Support Engineer)-->supe:sem(Support Engineering Manager);
```

```
supe:sem(Support Engineering Manager)-->supe:ssem(Senior Support Engineering Manager);
```

```
supe:ssem(Senior Support Engineering Manager)-->supe:ds(Director of Support);
```

end

```
click supe:se "/job-families/engineering/support-engineerssupport-engineer";
```

```
click supe:sse "/job-families/engineering/support-engineerssenior-support-engineer";
```

```
click supe:sem "/job-families/engineering/support-managementsupport-engineering-manager";
```

```
click supe:stse "/job-families/engineering/support-engineerstaff-support-engineer";
```

```
click supe:ssem "/job-families/engineering/support-management/senior-support-engineering-
manager"
```

```
click supe:ds "/job-families/engineering/support-managementdirector-of-support";
```

</div>

</div>

## Infrastructure Department

```
<div style="overflow-x: auto">
<div style="width: 1800px;">
```

```
mermaid
graph LR;
```

```
subgraph Merit Based
```

```
%% Backend Engineers
```

```
eng:abe(Intern, Backend Engineer)-->eng:be(Senior Backend Engineer);
```

```
eng:be(Backend Engineer)-->eng:sbe(Senior Backend Engineer);
```

```
%% Site Reliability Engineer
```

```
eng:asre(Intern, Site Reliability Engineer)-->eng:sre(Site Reliability Engineer);
```

```
eng:sre(Site Reliability Engineer)-->eng:srsre(Senior Site Reliability Engineer);
```

```
%% Database Reliability Engineer track
```

```
eng:adbre(Intern, Database Reliability Engineer)-->eng:dbre(Database Reliability Engineer);
```

```
eng:dbre(Database Reliability Engineer)-->eng:srdbre(Senior Database Reliability Engineer);
```

```
%% Infrastructure Analyst
```

```
eng:ia(Infrastructure Analyst)
```

```
end
```

```
subgraph Based on Merit & Company Need
```

```
%% Backend Engineers
```

```
eng:sbe(Senior Backend Engineer)-->eng:stbe(Staff Backend Engineer);
```

```
eng:stbe(Staff Backend Engineer)-->eng:pei(Principal Engineer Infrastructure);
```

```
eng:pei(Principal Engineer Infrastructure)-->eng:dei(Distinguished Engineer
```

```
Infrastructure);
```

```
eng:dei(Distinguished Engineer Infrastructure)-->eng:efi(Engineering Fellow
```

```
Infrastructure);
```

```
%% Site Reliability Engi
```